0051. N-Queens

Hard · Backtracking

leetcode.com/problems/n-queens

Problem

- Place n queens on an n x n board so none attack each other. Return all boards ("Q" and ".").

Intuition

- Exactly one queen per row, so recurse row by row and choose a column.
- A column is safe if no earlier queen shares its column, its "/" diagonal (r + c) or its "\" diagonal (r - c).
- Three boolean sets make each safety check O(1).

Brute Force

- Try all C(n^2, n) placements and validate: astronomically slow.

Optimal Approach

- Cols[], diag1[r+c], diag2[r-c+n-1].
- Place(r): if r == n record board.
- For c: if all three are free, mark, set board[r][c] = 'Q', place(r+1), unmark.

Worked Example

```
n = 4 (one solution path).

r0 c1  .Q..
r1 c3  ...Q
r2 c0  Q...
r3 c2  ..Q.   -> solution
(the other solution is the mirror: c2, c0, c3, c1)
```

Java Solution

```java
public class NQueens {
    public List<List<String>> solveNQueens(int n) {
        char[][] board = new char[n][n];
        for (char[] row : board) Arrays.fill(row, '.');
        List<List<String>> res = new ArrayList<>();
        place(0, n, board, new boolean[n], new boolean[2 * n], new boolean[2 * n], res);
        return res;
    }

    private void place(int r, int n, char[][] b, boolean[] cols, boolean[] d1, boolean[] d2, List<List<String>> res) {
        if (r == n) {
            List<String> sol = new ArrayList<>();
            for (char[] row : b) sol.add(new String(row));
            res.add(sol);
            return;
        }
        for (int c = 0; c < n; c++) {
            if (cols[c] || d1[r + c] || d2[r - c + n - 1]) continue;
            cols[c] = d1[r + c] = d2[r - c + n - 1] = true;
            b[r][c] = 'Q';
            place(r + 1, n, b, cols, d1, d2, res);
            b[r][c] = '.';
            cols[c] = d1[r + c] = d2[r - c + n - 1] = false;
        }
    }
}
```

Complexity

- Time: O(n!) roughly. Space: O(n^2) for the board.

Pitfalls

- Offset r - c by n - 1 so the index is non-negative.